Agent-to-Agent B2B Commerce Flow

Customer Grokbot ⇄ Store Grokbot ⇄ Supplier Grokbot

self-driving-store-b2b · Medusa 2 B2B Starter · Nov 2026

In one paragraph

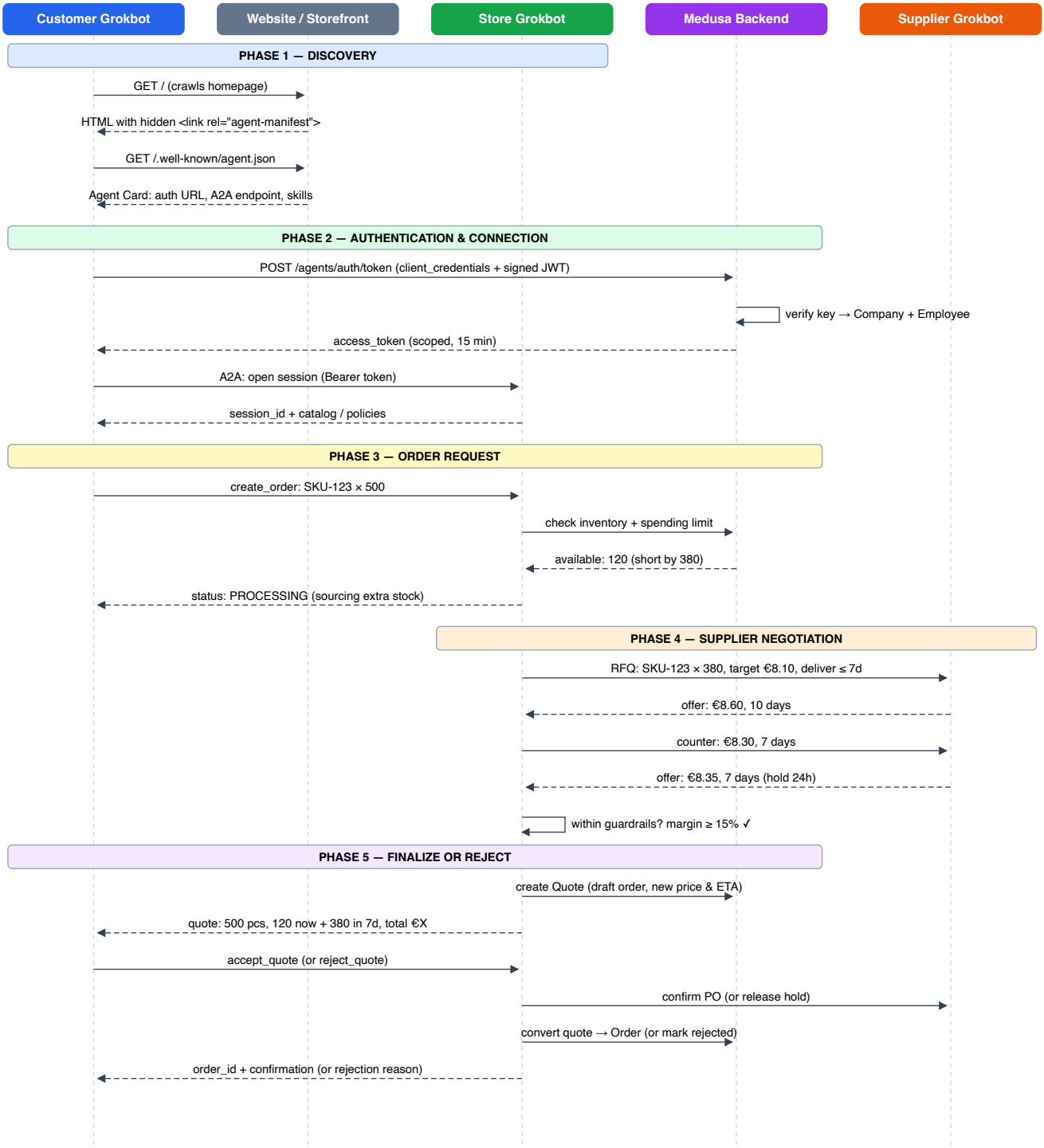
A buying company's AI agent (**Customer Grokbot**) visits the store website, finds a machine-readable entry point that is invisible to human visitors, authenticates as its company, and opens a session with the store's own AI agent (**Store Grokbot**). The Customer Grokbot can create orders. When it asks for more units than are in stock, the Store Grokbot contacts the **Supplier Grokbot**, negotiates price and delivery within preset limits, and then goes back to the Customer Grokbot with a quote. The order is then either finalized or rejected.

Actors

Actor	Owned by	Responsibility
Customer Grokbot	Buying company	Discovers the store, authenticates, browses, places orders, and accepts or rejects quotes.
Website / Storefront	Store (Next.js)	Serves the hidden discovery entry points (agent card, meta tags, robots hints).
Store Grokbot	Store	The store's sales agent. Talks to customer bots, checks stock and limits, sources from suppliers, and issues quotes.
Medusa Backend	Store	Source of truth: companies, employees, inventory, carts, quotes, approvals, and orders.
Supplier Grokbot	Supplier	Answers RFQs, negotiates, holds stock, and confirms purchase orders.

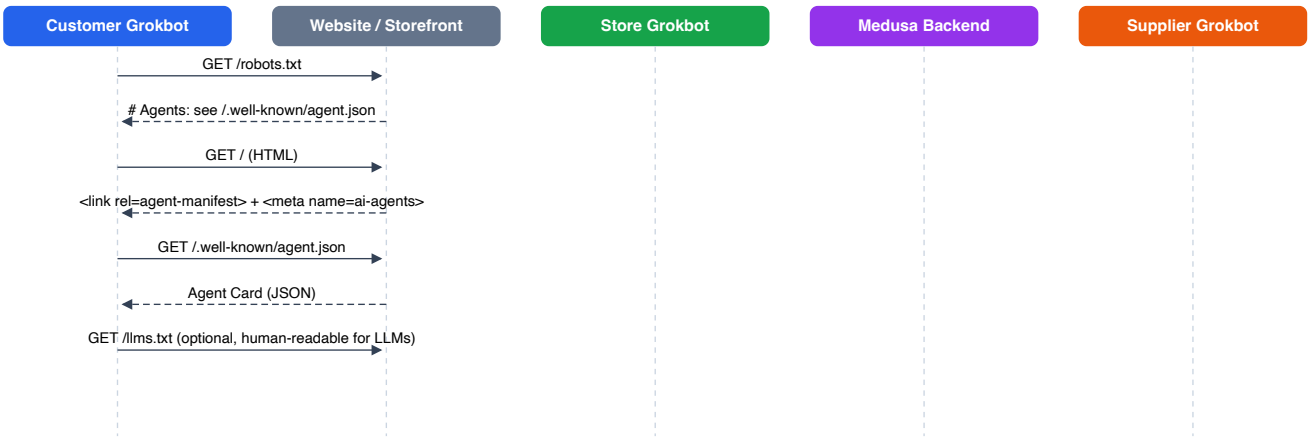
1. End-to-end sequence

The full happy path for an order that exceeds available stock. The five phases are described in detail on the following pages.



2. Phase 1: Discovery (hidden from people, visible to bots)

Humans browsing the site see nothing new. Agents find the entry point through standard, machine-readable signals. "Hidden" here means **not rendered in the UI**. It is **not** a security measure: anyone can read these files, so all protection happens in Phase 2.



Entry points (use several so any crawler finds one)

Where	What
`<head>` of every page	`<link rel="agent-manifest" href="/.well-known/agent.json">` and `<meta name="ai-agents" content="/.well-known/agent.json">`
`/robots.txt`	Comment line: `# AI agents: /.well-known/agent.json`
`/.well-known/agent.json`	Agent Card (A2A-style) describing auth, endpoint, and skills
`/llms.txt`	Plain-text instructions for LLM agents (optional)
HTTP header	`Link: </.well-known/agent.json>; rel="agent-manifest"` for bots that only read headers

Example Agent Card: `/.well-known/agent.json`

```
{
  "name": "Store Grokbot",
  "description": "B2B sales agent for self-driving-store",
  "version": "1.0",
  "endpoint": "https://api.store.example/agents/a2a",
  "authentication": {
    "type": "oauth2_client_credentials",
    "token_url": "https://api.store.example/agents/auth/token",
    "registration_url": "https://store.example/account/company/agents",
    "client_assertion": "private_key_jwt"
  },
  "skills": [
    { "id": "search_catalog",  "description": "Search products, prices, stock" },
    { "id": "create_order",    "description": "Create an order for the company" },
    { "id": "get_order_status","description": "Track order / quote state" },
    { "id": "accept_quote",    "description": "Accept a counter-offer quote" },
    { "id": "reject_quote",    "description": "Reject a counter-offer quote" }
  ],
  "policies_url": "https://store.example/agent-terms"
}
```

3. Phase 2: Authentication and connection

A bot is never anonymous. Each Customer Grokbot is bound to one **Company** and acts as one of its **Employees** (both modules already exist in this repo), so the company's spending limits and approval rules apply to it automatically.

One-time onboarding (done by a human)

1. The company admin logs into the storefront and opens **Account → Company → AI Agents**.
2. They register the bot by uploading its **public key** (or a JWKS URL) and choosing scopes and an optional per-agent spending limit.
3. The store returns a `client_id`. The private key never leaves the buyer.

Every session (done by the bot)

1. The bot signs a short-lived JWT with its private key and calls `POST /agents/auth/token`.
2. The backend verifies the signature, resolves `client_id → company_id + employee_id`, and issues a scoped **access token valid for 15 minutes**.
3. The bot opens an A2A session with the Store Grokbot at `/agents/a2a` using `Authorization: Bearer …`.
4. The Store Grokbot replies with a `session_id`, the company's price list, limits, and the terms the bot must follow.

```
POST /agents/auth/token
grant_type=client_credentials
client_id=agt_01JCUST...
client_assertion_type=urn:ietf:params:oauth:client-assertion-type:jwt-bearer
client_assertion=eyJhbGciOiJFUzI1NiIs...

200 OK
{ "access_token": "eyJ...", "expires_in": 900,
  "scope": "catalog:read orders:write quotes:write",
  "company_id": "comp_01J...", "employee_id": "emp_01J..." }
```

Scopes

Scope	Allows
catalog:read	Search products, prices, and stock levels
orders:write	Create carts and orders (subject to spending limit and approvals)
quotes:write	Accept or reject quotes produced by the Store Grokbot

4. Phase 3: Order request

The Customer Grokbot sends a structured order request. The Store Grokbot validates it against Medusa before making any commitment.

```
→ create_order
{ "session_id": "ses_9f2", "idempotency_key": "po-2026-11-0042",
  "items": [{ "sku": "SKU-123", "quantity": 500 }],
  "max_unit_price": 10.00, "need_by": "2026-11-20",
  "allow_partial": true, "allow_backorder_quote": true }
```

Store Grokbot decision logic

Check	Result	Action
Product exists and the company may buy it	No	Reject with reason `PRODUCT_UNAVAILABLE`
Within employee spending limit	No	Create the cart and route it to **Company Approval** (existing approval module)
Stock ≥ requested qty	Yes	Reserve inventory → create order → **CONFIRMED** (skip Phase 4)
Stock < requested qty	Short	Reserve available stock, reply `PROCESSING`, go to **Phase 4**

```
← response
{ "status": "PROCESSING", "request_id": "req_77a",
  "available_now": 120, "shortfall": 380,
  "message": "Sourcing 380 extra units from supplier. Quote within 15 min.",
  "callback": "a2a push notification on session ses_9f2" }
```

5. Phase 4: Supplier negotiation

The Store Grokbot authenticates to the Supplier Grokbot in the same way a customer authenticates to the store (the store is the *customer* in this relationship). It then runs a bounded negotiation.

Guardrails (set by the store owner in Medusa Admin)

Rule	Example
Minimum margin	≥ 15% over supplier unit price
Max supplier unit price	customer `max_unit_price` ÷ 1.15
Max lead time	Must meet the customer's `need_by` date
Max rounds	3 counter-offers, then stop
Max PO value without human	€10,000. Above this, a human must approve in Admin.
Allowed suppliers	Whitelisted supplier agent cards only

Message exchange

```
Store → Supplier   rfq      { sku: "SKU-123", qty: 380, target_price: 8.10, deliver_by: "2026-11-15" }
Supplier → Store   offer    { offer_id: "of_1", unit_price: 8.60, lead_days: 10 }
Store → Supplier   counter  { offer_id: "of_1", unit_price: 8.30, lead_days: 7 }
Supplier → Store   offer    { offer_id: "of_2", unit_price: 8.35, lead_days: 7, hold_until: "+24h" }
Store (internal)   evaluate → sell at 8.35 × 1.15 = 9.60  ≤ customer max 10.00 ✓  lead 7d ≤ need_by ✓  →
DEAL
```

The supplier's final offer is a **hold** and not yet a purchase. Stock is reserved for 24 hours so the store can go back to the customer without committing money. If the supplier cannot fulfil the order, or no deal is found within the guardrails, the flow skips to **REJECTED** (or offers a partial order of the 120 units in stock).

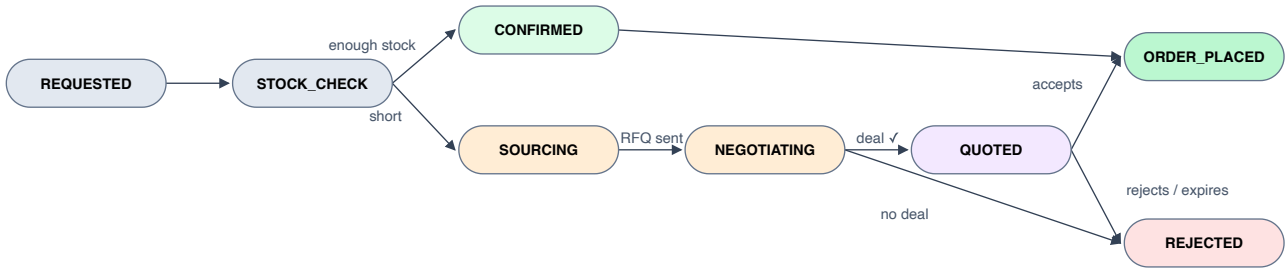
6. Phase 5: Finalize or reject

The Store Grokbot turns the result into a **Medusa Quote** (the existing quote module creates a draft order), so the whole deal is visible and auditable in Admin.

```
← quote (push to Customer Grokbot)
{ "quote_id": "quo_01J...", "expires_at": "2026-11-06T12:00:00Z",
  "lines": [
    { "sku": "SKU-123", "qty": 120, "unit_price": 9.00, "ships": "now" },
    { "sku": "SKU-123", "qty": 380, "unit_price": 9.60, "ships": "2026-11-13" }
  ],
  "total": 4728.00, "currency": "EUR",
  "options": ["accept_quote", "reject_quote", "accept_partial(120)"] }
```

Customer Grokbot replies	Store Grokbot → Supplier	Store Grokbot → Medusa	Final state
accept_quote	Confirm PO `of_2`	Quote accepted → converted to Order, inventory linked	**ORDER_PLACED**
accept_partial(120)	Release hold	Order for 120 units only	**ORDER_PLACED**
reject_quote	Release hold	Quote rejected, reservation released	**REJECTED**
No reply before `expires_at`	Release hold	Quote expired (scheduled job)	**REJECTED**

Order state machine



7. Security and safety

- **The discovery files are public.** Never put secrets in the agent card, meta tags, or llms.txt.
- **Signed client assertions** (private_key_jwt) are used instead of shared API keys. Keys are rotated and can be revoked per agent from the company page.
- **Short-lived, scoped tokens** are issued for each session and bound to company and employee, so existing spending limits and approvals always apply.
- **Idempotency keys** are required on `create_order` and `accept_quote` so retries never create duplicate orders.
- **Rate limits** apply per client_id and per company, plus a max order value per agent.
- **Prompt-injection defence:** bots exchange only **structured JSON**, and every free-text field is treated as data, never as instructions. Prices and quantities are validated by backend code, not by the LLM.
- **The LLM decides and code enforces.** Guardrails such as margin, max price, and max PO value are checked in Medusa workflows, so a bot cannot talk its way past them.
- **Human-in-the-loop** applies above value thresholds (merchant approval module) and to any new supplier.
- **Audit log:** every agent message, offer, and decision is stored and linked to the quote or order in Admin.

8. Mapping to this codebase (Medusa B2B starter)

Piece	Status	Where / what to build
Company, Employee, spending limits	**exists**	`backend/src/modules/company`
Quotes (draft order, accept/reject)	**exists**	`backend/src/modules/quote`
Company and merchant approvals	**exists**	`backend/src/modules/approval`
Hidden discovery entry points	**new**	`storefront/public/.well-known/agent.json` , `robots.txt` , `<head>` tags in root layout
Agent credentials module	**new**	`modules/agent-auth` : agent client, public key, scopes, link to employee
Token endpoint and auth middleware	**new**	`api/agents/auth/token` + middleware for `/agents/*`
A2A endpoint (Store Grokbot)	**new**	`api/agents/a2a` : LLM agent with tools that call Medusa workflows
Supplier module	**new**	`modules/supplier` : supplier, agent card URL, credentials, purchase orders, offers
Negotiation workflow	**new**	`workflows/source-from-supplier` with guardrail steps + compensation (release hold)
Quote expiry job	**new**	`jobs/expire-agent-quotes`
Admin UI	**new**	Agent registrations, guardrail settings, negotiation transcript widget on quote/order